

Сборщик мусора (GC) под микроскопом: поколения, режимы и тюнинг

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Екатерина Меттус

.NET разработчик

 @mettus_katerina

 mettus.ekaterina@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в чате учебной группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Режимы GC

Настройка GC

Управление задержками

Мониторинг GC

Рефлексия



Ваш бэкграунд?

Ваша цель на курсе?



Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

1. Настраивать режим работы GC (Server/Workstation)

Оптимизации под конкретный тип нагрузки

2. Использовать `WeakReference<T>`

Создание кэшей, устойчивых к давлению памяти

3. Диагностировать работу GC в реальном времени с помощью `dotnet-counters`

Контроль производительности

Режимы GC

Режимы GC

Режим Workstation

Режим Server

Режимы GC

Режим Workstation



Режим Server

Workstation

- Частая сборка мусора, зато работы меньше
- Пониженное потребление памяти
- Единственная управляемая куча
- Обычно используется в десктопных интерактивных приложениях

Режимы GC

Режим Workstation

Режим Server



Server

- Сборка мусора происходит реже
- Повышенная пропускная способность
- Повышенное потребление памяти
- Несколько управляемых куч
- Обычно используется в приложениях, параллельно обрабатывающих поступающие запросы

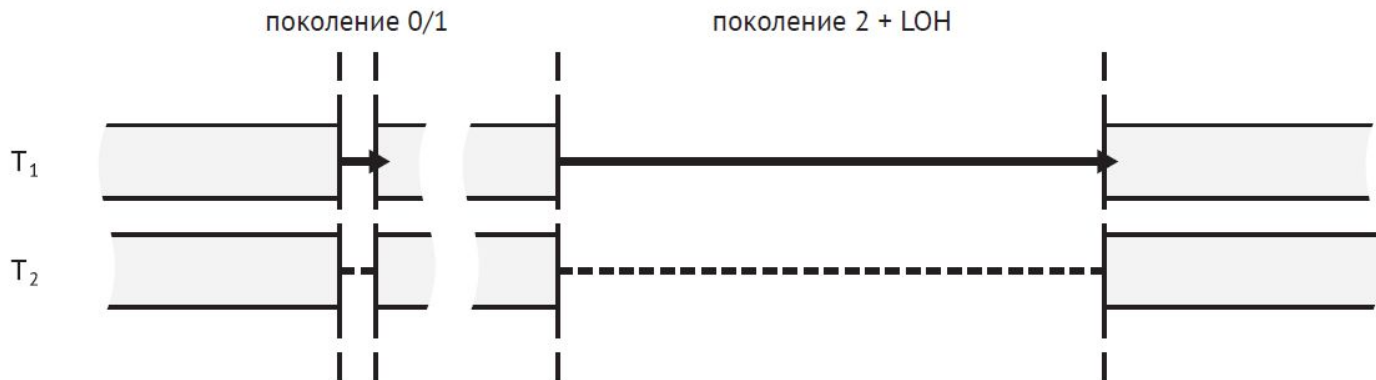
Режимы GC

Неконкурентный режим

Конкурентный режим

Неконкурентный workstation режим

- Все управляемые потоки приостанавливаются на все время сборки мусора независимо от того, в каком поколении она производится
- Код GC выполняется в потоке, инициировавшем сборку. При этом приоритет потока не изменяется, поток конкурирует с другими потоками в других приложениях
- При необходимости может выполняться уплотнение

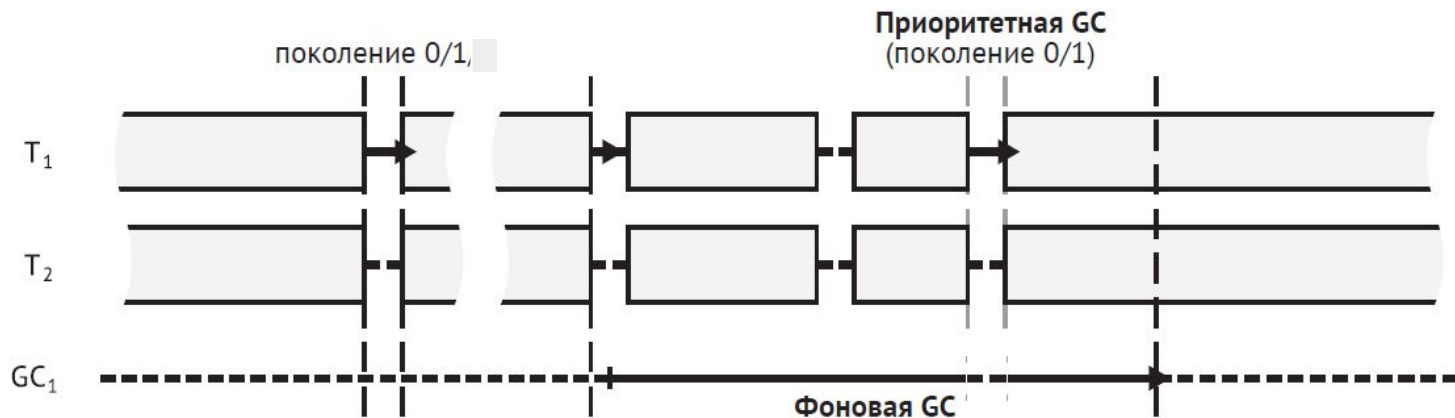


Сценарии применения

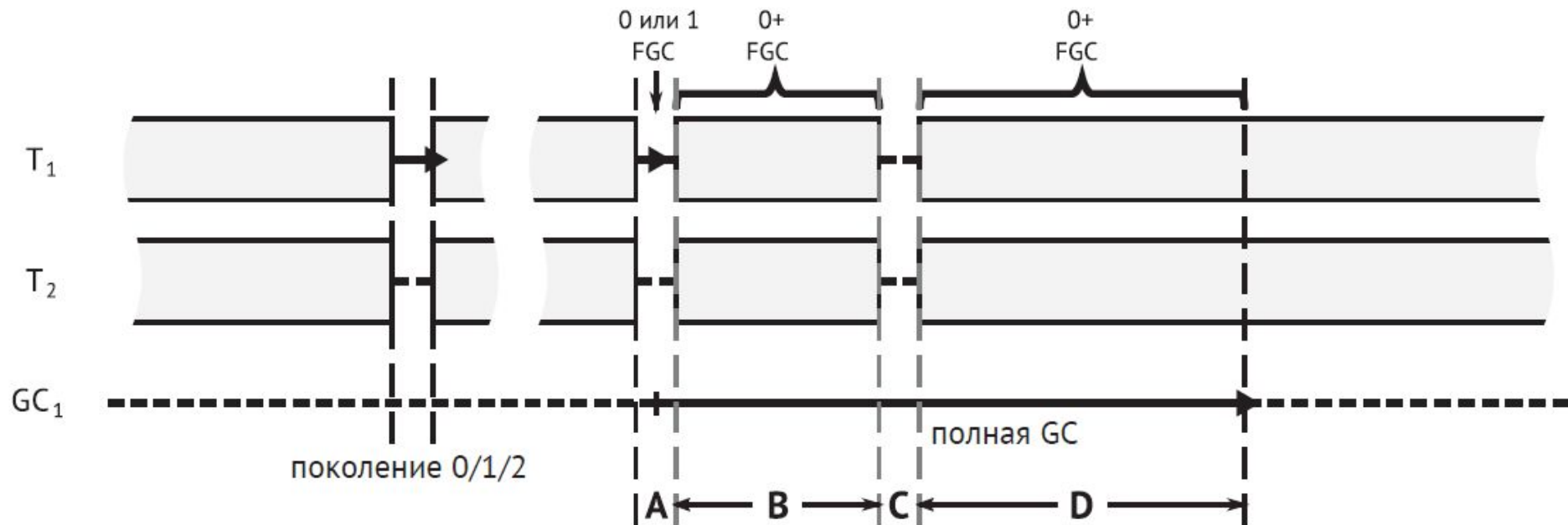
- В интенсивной среде, где ресурсов ЦП не хватает для обслуживания всех приложений, – так как нет специальных потоков GC, сборка мусора не увеличивает нагрузку на дефицитные процессорные ядра
- В среде, где работает много нетребовательных к ресурсам (потребляют мало памяти) веб-приложений (например, микросервисов в контейнере Docker).
Уменьшается общее количество потоков, что ценно для распределения процессорных ядер между большим количеством одновременно работающих приложений

Фоновый workstation режим

- Существует дополнительный поток, занимающийся исключительно сборкой мусора, большую часть времени он приостановлен в ожидании работы
- Сборка мусора в эфемерных поколениях неконкурентная, производится достаточно быстро. Это позволяет выполнить уплотнение при необходимости
- Полная сборка мусора может выполняться в двух режима: неконкурентном и фоновом



Фоновый workstation режим

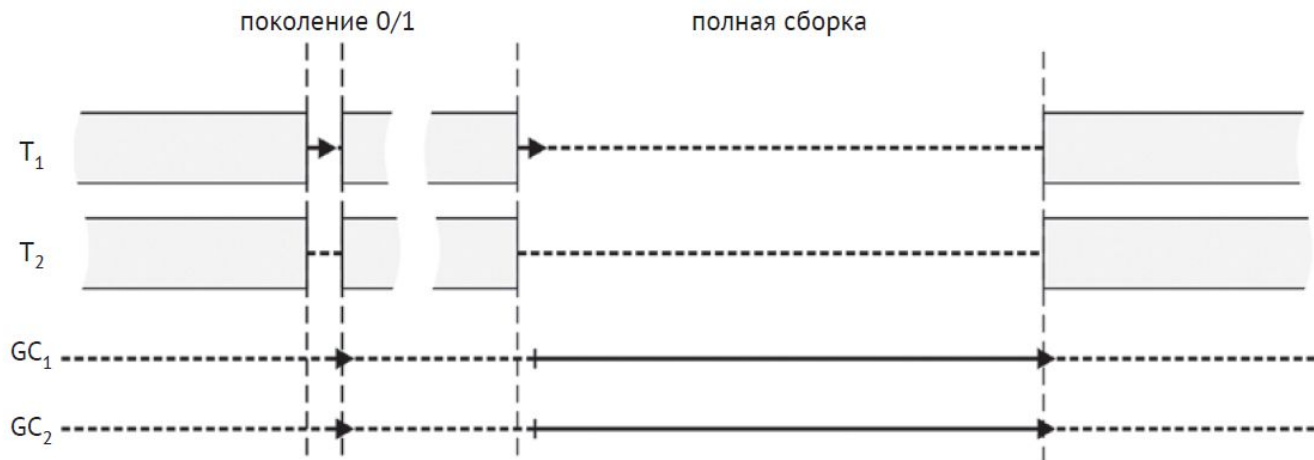


Сценарии применения

- Паузы короткие -> режим идеален для всех видов интерактивных приложений, большинство из которых составляют приложения с графическим интерфейсом

Неконкурентный серверный режим

- Существуют дополнительные потоки, предназначенные исключительно для сборки мусора, по умолчанию их ровно столько, сколько управляемых куч (серверные потоки сборки мусора). Большую часть времени они приостановлены в ожидании работы
- Все сборки мусора неконкурентные, может производиться уплотнение



Сценарии применения

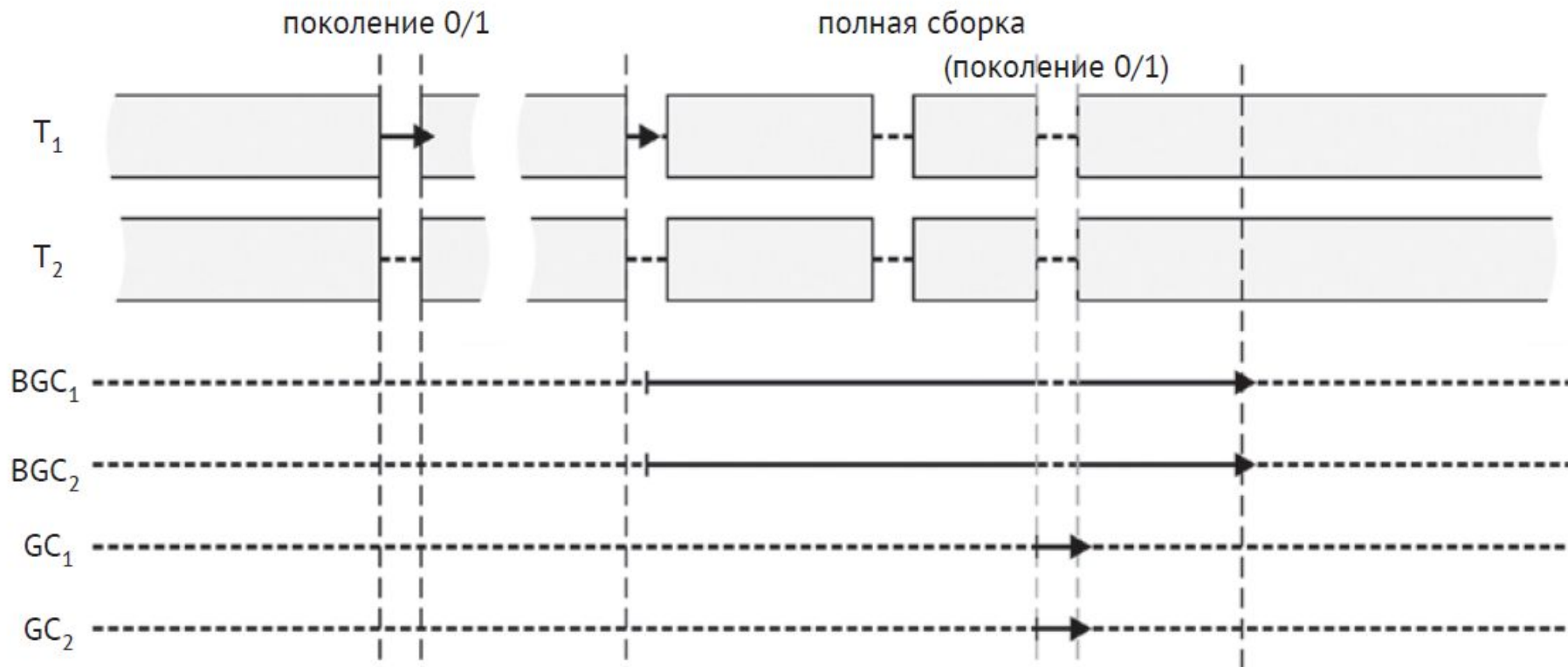
- В высоконагруженных веб-серверах, где присутствует сильная конкуренция за процессорные ядра
- Любая сборка мусора, включая полную, может быть уплотняющей, этот режим лучше борется с фрагментацией, чем конкурентный
- Все сборки блокирующие, не образуется плавающий мусор -> рабочий набор меньше



Фоновый серверный режим

- Для каждой управляемой кучи существует два потока, занимающихся исключительно сборкой мусора, – большую часть времени они приостановлены в ожидании работы:
 - серверные потоки GC – отвечают за выполнение всех блокирующих сборок мусора;
 - фоновые потоки GC – дополнительные потоки, по одному для каждой кучи, отвечающие за выполнение фоновых сборок мусора;
- Сборка мусора в поколениях 0 и 1 неконкурентная. Это позволяет выполнить уплотнение;
- Полная сборка мусора может производиться в двух режимах:
 - неконкурентный – все остальные потоки приостановлены, полная сборка может производиться с уплотнением.
 - фоновая GC – на протяжении большей части работы остальные управляемые потоки работают нормально. Уплотнение в этом режиме не производится.

Фоновый серверный режим



Сценарии применения

- Режим сборки мусора по умолчанию для большинства серверных приложений
- Ресурсоемкое настольное приложение, работающее на выделенной машине

Тип приложения	.NET Framework	.NET Core	.NET (5 и выше)
Консольные	Workstation GC; Concurrent GC	Workstation GC; Background GC	Workstation GC; Background GC
WinForms и WPF	Workstation GC; Concurrent GC	Workstation GC; Background GC	Workstation GC; Background GC
ASP.NET (IIS hosted)	Server GC; Background GC	—	—
ASP.NET Core	—	Server GC; Background GC	Server GC; Background GC
Worker Service / Generic Host	Workstation GC; Concurrent GC	Workstation GC; Background GC	Workstation GC; Background GC
Azure Functions	—	Server GC; Background GC	Server GC; Background GC

Настройка GC

Настройка

Для выбора режима GC можно
использовать:

- .csproj
- runtimeconfig.json или
runtimeconfig.template.json
- Переменные среды
(COMPlus_gcServer,
DOTNET_gcServer)

```
SomeApplication.runtimeconfig.json
{
  "runtimeOptions": {
    "tfm": "net8.0",
    "framework": {
      "name": "Microsoft.NETCore.App",
      "version": "8.0.0"
    },
    "configProperties": {
      "System.GC.Server": true,
      "System.GC.Concurrent": true
    }
  }
}
```

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <ServerGarbageCollection>true</ServerGarbageCollection>
    <ConcurrentGarbageCollection>>false</ConcurrentGarbageCollection>
  </PropertyGroup>
</Project>
```

Управление задержками

Управление задержками

`GCSettings.LatencyMode` — программно меняет режим задержек GC в рантайме.

Значения перечисления `GCLatencyMode`:

- `Batch` — максимальный throughput, минимальная отзывчивость.
- `Interactive (default)` — баланс между паузами и пропускной способностью.
- `LowLatency` — подавляет Gen 2 сборки, (используйте краткоременно!).
- `SustainedLowLatency` — длительная минимизация задержек, могут быть редкие блокировки.
- `NoGCRegion` — только через `GC.TryStartNoGCRegion()`.





LIVE



	Режим рабочей станции		Серверный режим	
	Неконкурентный	Конкурентный	Неконкурентный	Фоновый
Использование ЦП	Нет потоков GC	Один поток GC	Количество потоков GC равно количеству видимых логических ядер	Количество потоков GC в два раза больше количества видимых логических ядер
Batch	Да (по умолчанию)	Да (запрещены фоновые GC)	Да (по умолчанию)	Да (запрещены фоновые GC)
Interactive	Да (разрешены фоновые GC)	Да (по умолчанию)	Да (разрешены фоновые GC)	Да (по умолчанию)
LowLatency	Да	Да	Нет	Нет
Sustained-LowLatency	Нет	Да	Нет	Да
GCHeapCount	Нет	Нет	Да	Да
Типичное использование	Много облегченных приложений на одной машине, и все они терпимо относятся к длительным паузам (временем которых при необходимости можно управлять с помощью режима LowLatency)	Интерактивные приложения с жесткими требованиями ко времени реакции (при необходимости временем задержки можно управлять с помощью режимов LowLatency и SustainedLow-Latency)	В настоящее время встречается нечасто. Может рассматриваться как компромисс между ресурсоемким фоновым серверным режимом и фоновым режимом рабочей станции, в котором паузы на GC дольше. О длинных блокирующих GC можно сообщать с помощью уведомлений	Большинство приложений, занимающихся обработкой запросов (веб-приложения, размещенные в IIS, службы Windows)

Мониторинг GC

dotnet-counters

Инструмент для мониторинга и простой диагностики в реальном времени.

Отслеживает параметры: CPU, Heap Size, скорость аллокаций, счетчики GC, исключения и пр.

Работает на всех современных ОС, поддерживает x86, x64, ARM.



Команды

Установить:

```
dotnet tool install - global dotnet-counters
```

Получить список доступных процессов:

```
dotnet-counters ps
```

Запустить мониторинг:

```
dotnet-counters monitor -p 1234
```

Для экспорта:

```
dotnet-counters collect -p 1234 --format csv --output my_counters.csv
```



System.Runtime	
cpu-usage	The percent of process' CPU usage relative to all of the system CPU resources [0-100]
working-set	Amount of working set used by the process (MB)
gc-heap-size	Total heap size reported by the GC (MB)
gen-0-gc-count	Number of Gen 0 GCs between update intervals
gen-1-gc-count	Number of Gen 1 GCs between update intervals
gen-2-gc-count	Number of Gen 2 GCs between update intervals
time-in-gc	% time in GC since the last GC
gen-0-size	Gen 0 Heap Size
gen-1-size	Gen 1 Heap Size
gen-2-size	Gen 2 Heap Size
loh-size	LOH Size
alloc-rate	Number of bytes allocated in the managed heap between update intervals
assembly-count	Number of Assemblies Loaded
exception-count	Number of Exceptions / sec
threadpool-thread-count	Number of ThreadPool Threads
monitor-lock-contention-count	Number of times there were contention when trying to take the monitor lock between update intervals
threadpool-queue-length	ThreadPool Work Items Queue Length
threadpool-completed-items-count	ThreadPool Completed Work Items Count
active-timer-count	Number of timers that are currently active
Microsoft.AspNetCore.Hosting	
requests-per-second	Number of requests between update intervals
total-requests	Total number of requests
current-requests	Current number of requests
failed-requests	Failed number of requests

Оптимизация высоконагруженного сервиса

Ситуация: Сервис обрабатывал десятки тысяч запросов ежедневно, но пользователи сталкивались с медленными запросами из-за длительных пауз GC.

Решение:

Использовали dotnet-counters для мониторинга метрик GC, таких как количество сборок Gen 0/1/2 и их продолжительность.

Выявили, что частые сборки Gen 2 вызывали паузы до 1000 мс.

Оптимизировали код, минимизировав выделение больших объектов, и внедрили пулы объектов (ArrayPool).

Переключились на Server GC для увеличения пропускной способности.

Результат: Сокращение пауз GC с ~200 мс до ~60 мс и уменьшение процента медленных запросов на 40%.





LIVE



Слабые ссылки

WeakReference

WeakReference — слабая ссылка: объект может быть собран GC, если нет других ссылок.

Позволяет создавать "мягкие" кэши: данные хранятся "до первого давления памяти".

Применяется для объектов, чья инициализация дорогая, а потеря не критична.



ConditionalWeakTable

Специализированная коллекция ключ–значение для слабых ссылок на ключи.

Ключ хранится как слабая ссылка, значение — как сильная относительно ключа.
Значение автоматически удаляется после сборки ключа

Полезен для реализации кеширования или паттернов слабых событий



LIVE



Вопросы



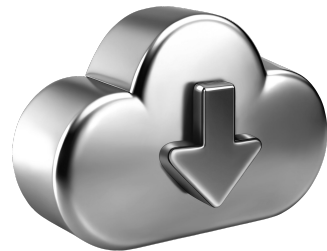
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Konrad Kokosa "Pro .NET Memory Management"
2. [Exploring .NET Performance: Practical Insights into dotnet-counters & dotnet-dump | by Mayura Gunasinghe | Medium](#)
3. [.NET garbage collection - .NET | Microsoft Learn](#)
4. [Debug a memory leak tutorial - .NET | Microsoft Learn](#)



**Подведём
итоги занятия**

Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

1. Настраивать режим работы GC (Server/Workstation)

Оптимизации под конкретный тип нагрузки

2. Использовать `WeakReference<T>`

Создание кэшей, устойчивых к давлению памяти

3. Диагностировать работу GC в реальном времени с помощью `dotnet-counters`

Контроль производительности



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️